

Documento de Olores de Software y Deuda Técnica

IC2101 – Programación Orientada a Objetos, I Semestre 2026

Nombre del equipo: Los Mogambos

Olores de software

1. Opacidad

Explicación de la estrategia usada

El nombre del método indica exactamente lo que hace, calcula la tabla de depreciación, evitando nombres ambiguos.

```
public String calcularTablaDepreciacion() {
    StringBuilder msg = new StringBuilder("Tabla de depreciación decreciente\n\n");
    double valorInicial = valor;
    double depreciacionAnual;
    double depreciacionAcumulada = 0.0;
    double valorFinal;

    String formatoEncabezado = "%-5s %-15s %-15s %-22s %-25s %-15s\n";
    String formatoFila = "%-5d $%-14.2f $%-14.2f $%-21.2f $%-24.2f $%-14.2f\n";

    msg.append(String.format(formatoEncabezado,"Año", "Valor inicial", "Tasa aplicada", "Depreciación anual", "Depreciación acumulada", "Valor final"));

    for (int anno = 1; anno <= vidaUtil; anno++) {
        depreciacionAnual = valorInicial * tasaDepreciacion;
        valorFinal = valorInicial - depreciacionAnual;

        if (valorFinal < valorSalvamento) {
            depreciacionAnual = valorInicial - valorSalvamento;
            valorFinal = valorSalvamento;
        }

        depreciacionAcumulada += depreciacionAnual;

        msg.append(String.format(formatoFila,
            anno, valorInicial, tasaDepreciacion, depreciacionAnual, depreciacionAcumulada, valorFinal));

        valorInicial = valorFinal;

        if (valorInicial <= valorSalvamento) {
            break;
        }
    }

    return msg.toString();
}

23
24 public String calcularTablaDepreciacion() {
25     StringBuilder msg = new StringBuilder("Tabla de depreciación lineal\n\n");
26     double valorInicial = valor;
27     double depreciacionAnual = valor / vidaUtil;
28     double depreciacionAcumulada = 0.0;
29     double valorFinal;
30
31     String formatoEncabezado = "%-5s %-15s %-22s %-25s %-15s\n";
32     String formatoFila = "%-5d $%-14.2f $%-21.2f $%-24.2f $%-14.2f\n";
33
34     msg.append(String.format(formatoEncabezado,"Año", "Valor inicial", "Depreciación anual", "Depreciación acumulada", "Valor final"));
35
36     for (int anno = 1; anno <= vidaUtil; anno++) {
37         depreciacionAcumulada += depreciacionAnual;
38         valorFinal = valorInicial - depreciacionAnual;
39
40         if (Math.abs(valorFinal) < 0.01) {
41             valorFinal = 0.0;
42         }
43
44         msg.append(String.format(formatoFila,
45             anno, valorInicial, depreciacionAnual, depreciacionAcumulada, valorFinal));
46
47         valorInicial = valorFinal;
48     }
49
50     return msg.toString();
51 }
```

2. Repetición innecesaria

Explicación de la estrategia usada:

Centralizamos la creación de campos de texto en esta clase para que los paneles no dupliquen el mismo código.

```
import java.awt.*;
import javax.swing.*;

public class ConstructorCampos{
    public static JTextField crearCampoTexto(int columnas) {
        JTextField campo = new JTextField(columnas);
        campo.setFont(new Font("Arial", Font.PLAIN, 16));
        campo.setForeground(Color.WHITE);
        campo.setBackground(new Color(45, 45, 45));
        campo.setCaretColor(Color.WHITE);
        return campo;
    }

    public static JPanel crearCampoConEtiqueta(String texto, JTextField campo) {
        JPanel contenedor = new JPanel();
        contenedor.setLayout(new BoxLayout(contenedor, BoxLayout.Y_AXIS));
        contenedor.setBackground(new Color(30, 30, 30));

        JLabel etiqueta = new JLabel(texto);
        etiqueta.setForeground(new Color(150, 150, 150));
        etiqueta.setFont(new Font("Arial", Font.PLAIN, 13));
        etiqueta.setAlignmentX(Component.LEFT_ALIGNMENT);

        campo.setAlignmentX(Component.LEFT_ALIGNMENT);

        contenedor.add(etiqueta);
        contenedor.add(Box.createVerticalStrut(6));
        contenedor.add(campo);
        return contenedor;
    }
}
```

3. Complejidad innecesaria

Explicación de la estrategia usada:

La validación resuelve la verificación en una sola línea sin pasos de más.

```

1 public class Validador {
2
3     public boolean esStringValido(String texto) {
4         return texto != null && !texto.trim().isEmpty();
5     }
6
7     public boolean esDoublePositivo(double valor) {
8         return valor > 0;
9     }
10
11     public boolean esIntPositivo(int valor) {
12         return valor > 0;
13     }
14
15     public boolean esDoubleNoNegativo(double valor) {
16         return valor >= 0;
17     }
18
19     public boolean esDoubleEnRango(double valor, double minIncluyente, double maxIncluyente) {
20         return valor >= minIncluyente && valor <= maxIncluyente;
21     }
22
23     public boolean esMenorQueEstricto(double valor, double limite) {
24         return valor < limite;
25     }
26 }

```

4. Inmovilidad

Explicación de la estrategia usada:

Esta clase es reutilizable en todos los paneles sin reescribir código de mensajes.

```

1 public class DialogoMensaje {
2
3     public static void mostrarError(Component padre, String mensaje) {
4         JOptionPane.showMessageDialog(
5             padre,
6             mensaje,
7             "Error",
8             JOptionPane.ERROR_MESSAGE);
9     }
10
11     public static void mostrarExito(Component padre, String mensaje) {
12         JOptionPane.showMessageDialog(
13             padre,
14             mensaje,
15             "Éxito",
16             JOptionPane.INFORMATION_MESSAGE);
17     }
18
19     public static void mostrarAdvertencia(Component padre, String mensaje) {
20         JOptionPane.showMessageDialog(
21             padre,
22             mensaje,
23             "Advertencia",
24             JOptionPane.WARNING_MESSAGE);
25     }
26
27     public static boolean confirmar(Component padre, String mensaje) {
28         int opcion = JOptionPane.showConfirmDialog(
29             padre,
30             mensaje,
31             "Confirmación",
32             JOptionPane.YES_NO_OPTION);
33     }
34 }

```

5. Viscosidad

Explicación de la estrategia usada:

Usamos siempre la misma clase Ordenamiento, así seguir el patrón correcto es tan fácil como copiarlo

```
public ArrayList<Responsable> listarResponsablesAlfabeticamente() {  
    ArrayList<Comparable> temporal = new ArrayList<Comparable>();  
  
    for (Responsable r : responsables) {  
        temporal.add(r);  
    }  
  
    Ordenamiento.ordenar(temporal);  
  
    ArrayList<Responsable> resultado = new ArrayList<Responsable>();  
  
    for (Comparable obj : temporal) {  
        resultado.add((Responsable) obj);  
    }  
  
    return resultado;  
}
```

```
public ArrayList<Maquinaria> listarMaquinariasPorVidaUtil() {  
    ArrayList<Comparable> temporal = new ArrayList<Comparable>();  
  
    for (Maquinaria m : maquinarias) {  
        temporal.add(m);  
    }  
  
    Ordenamiento.ordenar(temporal);  
  
    ArrayList<Maquinaria> resultado = new ArrayList<Maquinaria>();  
  
    for (Comparable obj : temporal) {  
        resultado.add((Maquinaria) obj);  
    }  
}
```

6. Rigidez

Explicación de la estrategia usada:

Separar en paquetes por responsabilidad permite cambiar una parte sin afectar las demás.

```

2
3 import java.awt.*;
4 import javax.swing.*;
5 import proyecto.model.Maquinaria;
6 import proyecto.model.Responsable;
7 import proyecto.model.Sistema;
8 import proyecto.util.Validador;
9 import proyecto.view.gui.Depreciacion.PanelTablasPorResponsable;
10 import proyecto.view.gui.dialogos.DialogoMensaje;
11 import proyecto.view.gui.maquinarias.ConsultarDetalladamente;
12 import proyecto.view.gui.maquinarias.ListarTodas;
13 import proyecto.view.gui.maquinarias.PanelRegistrarEquipoIndustrial;
14 import proyecto.view.gui.maquinarias.PanelRegistrarHerramientaIndustrial;
15 import proyecto.view.gui.operaciones.AsignarResponsable;
16 import proyecto.view.gui.operaciones.MauinariaAsignadaResponsable;
17 import proyecto.view.gui.operaciones.PanelRegistrarMantenimiento;
18 import proyecto.view.gui.responsables.ListarResponsable;
19 import proyecto.view.gui.responsables.PanelConsultarResponsable;
20 import proyecto.view.gui.responsables.PanelEliminarResponsable;
21 import proyecto.view.gui.responsables.PanelRegistrarResponsable;
22
23 public class GUIPrincipalProyecto {
24     public static void main(String[] args) {
25
26         JFrame ventana = new JFrame("Gestión de Maquinaria");
27         ventana.setSize(1150,700);
28         ventana.setResizable(false);
29         ventana.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
30         ventana.setLocationRelativeTo(null);
31         ventana.setLayout(new BorderLayout()); // 5 secciones
32

```

7. Fragilidad

Explicación de la estrategia usada:

Sistema no accede directamente a la lista interna de Responsable, así un cambio ahí no rompe a Sistema.

```

public boolean eliminarResponsable(String pIdentificacion) {
    Responsable r = buscarResponsablePorIdentificacion(pIdentificacion);

    if (r == null) {
        return false;
    }

    while (r.cantidadMaquinarias() > 0) {
        r.removeMaquinaria(r.getMaquinarias().get(0));
    }

    responsables.remove(r);
    return true;
}

```

Deuda técnica

¿Qué es la deuda técnica?

La deuda técnica es cuando se elige una solución rápida o fácil en el momento, en lugar de la solución correcta, y eso genera más trabajo o más errores después. Es como una deuda, no se paga el costo ahora, pero lo vamos a pagar con intereses más adelante.

Decisiones tomadas por el equipo para evitarla:

No duplicamos lógica de ordenamiento: se implementó una sola vez y se reutiliza para responsables y maquinarias.

```

1 package proyecto.model;
2
3 public interface Comparable {
4
5     public boolean menorQue(Comparable pObj);
6
7 }

```

```

import java.util.ArrayList;
import proyecto.model.Comparable;

public class Ordenamiento {

    public static void ordenar(ArrayList<Comparable> arreglo) {
        int n = arreglo.size();
        for (int i = 0; i < n - 1; i++) {
            for (int j = 0; j < n - i - 1; j++) {
                if (arreglo.get(j + 1).menorQue(arreglo.get(j))) {
                    Comparable temp = arreglo.get(j);
                    arreglo.set(j, arreglo.get(j + 1));
                    arreglo.set(j + 1, temp);
                }
            }
        }
    }
}

```